

Esercizio: Conway Audioactive Decay con Processi, Pipe e Memoria Condivisa

Viene fornito il file `audioactive.c`, che implementa in modo sequenziale il calcolo del *Conway Audioactive Decay*.

Il decadimento audioattivo di Conway è una sequenza ottenuta descrivendo verbalmente le cifre della stringa corrente. Ad esempio:

- $1 \rightarrow 11$ (“un 1”)
- $11 \rightarrow 21$ (“due 1”)
- $21 \rightarrow 1211$ (“un 2, un 1”)
- $1211 \rightarrow 111221$ (“un 1, un 2, due 1”)

Il programma `audioactive.c` fornito:

- riceve da linea di comando:
 - un seme numerico positivo
 - il numero di iterazioni
- calcola iterativamente il decadimento audioattivo
- stampa ogni iterazione su `stdout`

Per la compilazione: `gcc audioactive.c -o audioactive`

Per il test: `.\audioactive <seme> <iterazioni>`

Esempi:

```
./audioactive 1 3
11
21
1211

./audioactive 3 5
13
1113
3113
132113
1113122113
```

Obiettivo

Realizzare una versione multiprocesso del programma utilizzando:

- `fork()`
- `pipe()`
- memoria condivisa anonima (`mmap()` con `MAP_SHARED | MAP_ANONYMOUS`)

L'esercizio è suddiviso in due step incrementali.

Step 1 — Pipeline di processi con pipe

Creare un nuovo programma multiprocesso in cui:

1. Il processo padre crea:
 - un numero di figli pari al numero di iterazioni richieste.
 - un numero di pipe sufficiente per creare una comunicazione ad anello (dal padre al primo figlio, dal primo figlio al secondo figlio, ... dal penultimo figlio all'ultimo figlio).
2. Il padre invia il seme al primo figlio tramite una **pipe**.
3. Ogni figlio:
 - riceve una stringa tramite **pipe**
 - calcola l'audioactive decay sulla stringa ricevuta, ottenendo la stringa risultato
 - stampa il risultato su **stdout**
 - invia il risultato al figlio successivo tramite una **pipe**
 - termina
4. Il padre attende la terminazione di tutti i figli.

Vincoli

Ogni figlio deve mantenere aperti solo i descrittori di file necessari.

L'output generato dal programma modificato deve essere identico a quello prodotto dal programma `audioactive.c`

Il programma deve avere nome `audioactive-pipe.c`

Step 2 — Memoria condivisa anonima

Estendere la soluzione dello Step 1 utilizzando memoria condivisa anonima.

Il padre deve creare:

1. un contatore condiviso inizializzato a 0
2. un array condiviso di 1 MB (`char shared_buffer[1000000]`)

Comportamento richiesto

Ogni figlio:

1. riceve la stringa dalla **pipe**
2. calcola la stringa risultato
3. **non deve stampare nulla**

4. copia il risultato, seguito dal carattere `\n`, nell'array condiviso a partire dalla posizione indicata dal contatore condiviso
5. incrementa il contatore condiviso del numero di caratteri copiati
6. inoltra il proprio risultato al figlio successivo
7. termina

Il padre: 1. invia il seme al primo figlio 2. attende che tutti i figli terminino 3. stampa su `stdout` il contenuto dell'array condiviso

Vincoli

Ogni figlio deve mantenere aperti solo i descrittori di file necessari.

L'output generato dal programma modificato deve essere identico a quello prodotto dal programma `audioactive.c`

Il programma deve avere nome `audioactive-pipe-shm.c`